STAT214 Chapter 3 Setting Up Your Data Science Project

1 Programming Languages and IDEs

1.1 Programming Languages

- **Python**: well suited for ML projects, particularly natural language processing (NLP) and image classification problems
- **R**: has a range of excellent packages for statistical analysis and the analysis of medical and biological data, and it is particularly well suited for data visualization

1.2 IDE

- **Python**: Jupyter Notebook IDE, Visual Studio Code
- **R**: RStudio IDE

1.3 Code Guidelines

- Do not repeat yourself
- Follow a consistent style guide ([“tidyverse” style guide](), [PEP8 style guide]())
- Comment liberally
- Test your code
- Conduct code review

1.4 Writing Functions for Data Cleaning and Preprocessing

It is strongly recommend that you write a function that

- takes in the original, raw data as its input
- returns a cleaned/preprocessed version of the data as the output.

---

2 A Consistent Project Structure

2.1 File Format

- **Python**: writing your exploration and analysis code in Jupyter Notebook
- **R***: writing your exploration and analysis code in Quarto with RStudio
- Recommend saving the code for reusable functions (such as functions for data cleaning and preprocessing) in **scripts**

2.2 An Example File Structure (Using R)

```
data/
  data_documentation/
    codebook.txt
  data.csv
dslc_documentation/
  01_cleaning.qmd
  02_eda.qmd
  03_clustering.qmd
  04_prediction.qmd
  functions/
    cleanData.R
    preProcessData.R
README.md
```

- `data/`
  - raw dataset

    - data documentation information
- `dslc_documentation/`
    - separate `.qmd` Quarto file (for R) or `.ipynb` Jupyter Notebook (for Python) for conducting and documenting the code-based explorations and analysis at each stage of the DSLC (with numeric prefix)
    - `functions/` subfolder with the `.R` (R) or `.py` (Python) scripts
- `README.md`

---

3 Reproducibility

3.1 Several Types of Reproducibility

In increasing order of the strength of the evidence of trustworthiness that is provided:

- demonstrating that the results can be regenerated by rerunning the original code on the same data on the same computer
- reproduce the results using the same data but new code written by a different person
- an entirely independent team reproducing a result by collecting entirely new data and writing their own code to conduct their own analysis

3.2 Tips and Tricks for Reproducibility

- Avoid handling raw data or doing manual modifications
- Regularly rerun your code from start to finish in a fresh R/Python session
- Tidy up your code files
- Use version control (e.g., Git)
- Document everything
- Use up-to-date software